Data-driven Koopman mode approximation: A neural power iteration algorithm

Guillaume O. Berger[1], and Raphaël M. Jungers[1]

***Abstract*— This paper proposes a novel data-driven algorithm to approximate the dominant eigenfunctions (aka. modes) of the Koopman operator of nonlinear dynamical systems using neural networks. The relevance of learning the dominant Koopman modes is to approximate nonlinear dynamics by linear ones in a lifted space, thereby enabling simplified control and analysis. To fight the curse of dimensionality arising from using expressive templates (here neural networks) for the mode approximation, the proposed method leverages a power-iteration scheme that directly learns the dominant Koopman modes without explicitly constructing the projection of the Koopman operator on the template of functions. Our approach connects to other approaches in the literature that avoid the curse of dimensionality by learning small dictionaries of functions, but differs from them in that we do not require "anti-collapse mechanisms" to ensure that the learned dictionary is expressive enough to approximate the Koopman operator since our power-iteration scheme is designed to converge toward the dominant modes of the projected Koopman operator. The approach is fully data-driven, requiring only sampled state transitions. Theoretical guarantees are provided, showing convergence under increasing sample size and network width (in connection with the neural tangent kernel theorem). Numerical experiments demonstrate that the method achieves accurate and smooth approximations of dominant modes while avoiding the limitations of traditional techniques such as extended dynamic mode decomposition.**

I. Introduction

Koopman theory [1] offers a powerful alternative to state-based techniques for studying nonlinear dynamical systems

$$x^+ = F(x).$$

Rather than studying the evolution of state variables $x \in \mathbb{R}^n$, the Koopman approach studies the evolution of observables, i.e., functions $f : \mathbb{R}^n \to \mathbb{C}$, under the action of the system. The overarching idea of this approach is that the action of the system on the observables can be represented by a linear operator, called *Koopman operator*, defined by

$$\mathcal{K} : \mathcal{F} \to \mathcal{F}, \quad (\mathcal{K}f)(x) = f(F(x)),$$

where $\mathcal{F}$ is a suitable set of observables. This allows to apply techniques from linear operator theory, such as spectral analysis, to study the system, including convergence analysis [2], chaos [3], model order reduction [4], and control [5]; see [6]–[9] for recent surveys.

[1]Both authors are with ICTEAM, UCLouvain, Louvain-la-Neuve, Belgium. GB is an FNRS Postdoctoral Researcher. RJ is an FNRS honorary Research Associate. This project has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme under grant agreement no. 864017 (L2C), from the Horizon Europe programme under grant agreement no. 101177842 (Unimaas), and from the ARC (French Community of Belgium) (SIDDARTA). Emails: {guillaume.berger,raphael.jungers}@uclouvain.be.

Another key advantage of the Koopman approach is that it is easily applicable in the data-driven setting. Indeed, the action of the system on an observable f can be approximated by its action on a subset of sample states $\{x_i\}_{i=1}^N \subseteq \mathbb{R}^n$, which gives a straightforward data-driven approximation

$$(\mathcal{K}f)(x_i) = f(x_i^+), \quad i = 1, \dots, N,$$

where $\{(x_i, x_i^+)\}_{i=1}^N$ are sample one-step trajectories of the system.

However, the set $\mathcal{F}$ of observables being typically infinite-dimensional, Koopman-based techniques usually require projection on a finite-dimensional subspace [10]. More precisely, given a finitely-generated subspace $\mathcal{D} \subseteq \mathcal{F}$, one studies the projected Koopman operator

$$K_{\mathcal{D}} : \mathcal{D} \to \mathcal{D}, \quad K_{\mathcal{D}} = \Pi_{\mathcal{D}} \circ \mathcal{K},$$

where $\Pi_{\mathcal{D}}$ is a projection on $\mathcal{D}$. The projection introduces approximation errors that need to be taken into account in the analysis of the system through the projected Koopman operator. This raises the fundamental question of the selection of the projection subspace $\mathcal{D}$, which needs to balance computational efficiency and accuracy of the approximation [11].

The dynamic mode decomposition (DMD) [12] approach uses the set of linear observables as space $\mathcal{D}$. The extended DMD (EDMD) [13] extends the former with a user-provided dictionary of functions (e.g., polynomials of degree $\leq d$) as basis of $\mathcal{D}$. The Hankel DMD (aka. time-delay DMD) [14] is a particular case of EDMD where the dictionary is built from a user-provided function g on which the Koopman operator is applied m times.

The reliance on a user-provided dictionary is a significant limitation because good approximations of the Koopman operator typically require expressive dictionaries. For instance, bistable systems have discontinuous dominant eigenmodes of their Koopman operator [2]; thereby requiring polynomials of high degree for good approximations of such modes.

Consequently, in recent years, there has been significant research effort devoted to exploiting the expressive power of (deep) neural networks to parametrize observables [15]–[19]. The main challenge with this approach is that there is no clear basis of the associated space $\mathcal{D}$. The learning problem is therefore formulated as the one of finding a dictionary of neural network observables along with the associated projected Koopman operator:

$$\min_{\substack{\theta_i \in \mathbb{R}^{n_\theta} \\ K \in \mathbb{C}^{m \times m}}} \|[\mathcal{K}\phi^{\theta_1}, \dots, \mathcal{K}\phi^{\theta_m}] - [\phi^{\theta_1}, \dots, \phi^{\theta_m}]K\|, \quad (1)$$

where $\phi^\theta \in \mathcal{F}$ is a neural network parametrized by $\theta \in \mathbb{R}^{n_\theta}$, and K is the matrix representation of the projected Koopman operator associated to $\mathcal{D} = \text{span}\{\phi^{\theta_1}, \dots, \phi^{\theta_m}\}$. Although attractive for its simplicity, formulation (1) is problematic because of the *collapse problem*: generically, the dictionary collapses to a trivial dictionary with only a few linearly independent observables (e.g., $\phi^{\theta_i} = 0$ for all i) [15], [16].

To prevent collapse, state-of-the-art approaches (see refs above) require that the state can be (approximately) retrieved from the encoding $[\phi^{\theta_1}(x), \dots, \phi^{\theta_m}(x)]$. This is enforced either explicitly by fixing some of the observables ϕ^{θ_i} so that x can be retrieved from those observables [15], or implicitly by adding a penalty term in the loss of (1):

$$\rho \|\text{Id} - \Delta^{\theta_0} \circ [\phi^{\theta_1}, \dots, \phi^{\theta_m}]\|,$$

where Id is the identity function and $\Delta^\theta : \mathbb{C}^m \to \mathbb{R}^n$ is a trainable neural network playing the role of the *decoder* [16]–[19].[1] The drawback of these state-retrieval approaches is that they require a number of observables that grows with the dimension of the system, thereby limiting their use in systems of large dimension. Furthermore, the decoder-based approach requires to train an additional network, which makes training slower and more data-sensitive.

Contributions

In this work, we propose a novel iterative method, based on power iterations, to learn the dominant modes of the projected Koopman operator. A *mode* of the projected Koopman operator is an eigenfunction of it, that is, a non-zero observable $g \in \mathcal{D}$ such that $K_\mathcal{D} g = \lambda g$ for some $\lambda \in \mathbb{C}$ called the associated eigenvalue. A mode is *dominant* if its eigenvalue is large in modulus compared to the other eigenvalues. Power iteration is a numerical iterative algorithm that allows one to approximate the m dominant modes of a linear operator, where m is a user-provided parameter [20].

Conceptually, our approach works as follows: starting from a set of observables $\{f_1, \dots, f_m\}$, we apply recursively $K_\mathcal{D}$ to each observable f_i. Under appropriate assumptions (e.g., separation of the dominant m eigenvalues of $K_\mathcal{D}$ [20]), the set $\{K_\mathcal{D}^k f_1, \dots, K_\mathcal{D}^k f_m\}$ converges toward a basis of the subspace spanned by the dominant m modes of $K_\mathcal{D}$. To avoid ill-conditioning, a normalization step can be applied periodically. The above conceptual approach is made practical by (i) using data to approximate the action of the exact Koopman operator, and (ii) using gradient descent to project back on the set of neural-network observables.

Concretely, the contributions of this work are as follows:

- **Theory:** We provide the first power-iteration-based algorithm to approximate the dominant modes of the projected Koopman operator, without requiring its explicit construction; thereby alleviating the need of large user-provided dictionaries, or anti-collapse mechanisms for learned dictionaries;
  We discuss the soundness of our approach and of its data-driven neural-network-based implementation (see below), based on the neural tangent kernel theorem [21];
- **Implementation:** We provide a tractable data-driven implementation of the above algorithm for observables parametrized by neural networks;
- **Experiments:** We demonstrate the usefulness of our approach on numerical examples.

[1] In other words, we impose that $x \approx \Delta^{\theta_0}(\phi^{\theta_1}(x), \dots, \phi^{\theta_m}(x))$.

Applications of Koopman mode decomposition

The Koopman mode decomposition (KMD), introduced by Mezić [4], exploits the spectral properties of the Koopman operator to decompose observables of nonlinear dynamical systems into components with simple linear evolution; see also [2], [6], [22] for surveys. Concretely, if $f = \sum_{i=1}^\infty \alpha_i g_i$, where g_i are eigenfunctions of $\mathcal{K}$ with eigenvalues λ_i, then $\mathcal{K}^k f = \sum_{i=1}^\infty \alpha_i \lambda_i^k g_i$. Such an eigenfunction expansion is not available for every dynamical system or every observable space: the point spectrum may be insufficient or even absent, and Koopman eigenfunctions need not form a basis [3]. The KMD, if it exists, however shows that only the dominant modes will dictate the long-term dynamic (i.e., the value of $\mathcal{K}^k f$ for large k) because λ_i^k will converge faster to zero for the dominated modes than for the dominant ones. Notably, the modes with eigenvalue one give the basins of attraction of the system, while the sub-unitary dominant eigenvalues give the rate of convergence of the system toward attractors [2], [22]; see also Section II.

It is also well known that the projection of the Koopman operator on a subspace introduces perturbations in the spectrum of $\mathcal{K}$. This needs to be taken into account in the analysis of the system using the projected Koopman operator; see, e.g., [10], [11]. At the level of $\mathcal{K}$ itself, Generalized Laplace Analysis (GLA) [4] provides a related spectral-extraction viewpoint based on weighted time averages; notably, the averaging can separate eigenvalues of equal modulus but different phases, which ordinary power iteration cannot. Here, however, our goal is to learn the dominant modes of the projected operator $K_\mathcal{D}$ rather than analyze the spectrum of $\mathcal{K}$ itself. We therefore leave these spectral-approximation questions aside, while noting that they motivate the use of expressive sets $\mathcal{D}$.

Control can also be achieved with Koopman mode decomposition when applied to the Koopman operator of open-loop systems [5], [7]. This will be left for future work as well.

Other related work

Recently, several research efforts in the field of machine learning and artificial intelligence have been devoted to learning the dynamics of "systems" or environments in reduced-order latent space representations allowing for prediction, analysis, and planning [23]–[25]. For instance, the joint embedded predictive architecture (JEPA) learns together a latent representation of the state of the environment using neural network encoders as well as a neural network predictive model for the evolution of the latent representation over time. These approaches connect to the philosophy of this work in that they also aim to learn simultaneously the

latent representation of the state and the predictive model in the latent space. Consequently, like us, they face the same risk of collapse of their model during training. The differences with our approach are twofold: (i) they use neural network as predictive models, whereas we use linear models (encoded by the matrix $K_{\mathcal{D}}$); and (ii) they fight collapse with techniques different from our power-iteration-based approach (e.g., contrastive samples, entropy maximization, decoders, etc.) [26].

Notation

Throughout this paper, we fix $n \in \mathbb{N}_{>0}$ and a *state space* $\mathcal{X} \subseteq \mathbb{R}^n$. Let $(\mathcal{X}, \Sigma, \mu)$ be a probability space on $\mathcal{X}$, with σ-algebra Σ and probability measure μ. We let $\mathcal{F}$ be the space $L^2(\mathcal{X}, \Sigma, \mu)$ endowed with the norm $\|\cdot\|$.[2] Note that $\mathcal{F}$ is a Hilbert space with inner product $\langle f, g \rangle \triangleq \mathbb{E}_\mu[f\bar{g}]$ [27]. We denote by i the imaginary unit.

A *feed-forward neural network* from $\mathbb{R}^n$ to $\mathbb{C}$ is generically denoted by ϕ^θ, where $\theta \in \mathbb{R}^{n_\theta}$ is a parameter vector for the weights and biases of the network. Concretely, for a number $L \in \mathbb{N}_{>0}$ of hidden layers with respective widths $c_1, \dots, c_L \in \mathbb{N}_{>0}$ and an activation function $\sigma : \mathbb{R} \to \mathbb{R}$, the network ϕ^θ is defined by

$$\begin{aligned} y_0 &= x, \\ y_{k+1} &= \sigma(W_k^\theta y_k + b_k^\theta), \quad k = 0, \dots, L-1, \\ \phi^\theta(x) &= [1, \mathrm{i}](W_L^\theta y_L + b_L^\theta), \end{aligned}$$

where the weight matrices $W_k^\theta \in \mathbb{R}^{c_{k+1} \times c_k}$ and bias vectors $b_k^\theta \in \mathbb{R}^{c_{k+1}}$ (with $c_0 = n$ and $c_{L+1} = 2$) are parametrized by θ, and σ is applied *entrywise* on vectors.

II. Problem statement

We consider a discrete-time dynamical system of the form

$$x(t+1) = F(x(t)), \quad t \in \mathbb{N},$$

where $F : \mathcal{X} \to \mathcal{X}$ is a continuous map. In particular, $\mathcal{X}$ is assumed to be forward invariant under F, so that $F(x) \in \mathcal{X}$ for every $x \in \mathcal{X}$. We assume throughout that $f \circ F \in \mathcal{F}$ for every $f \in \mathcal{F}$, so that the composition operator induced by F maps $\mathcal{F}$ into itself.

Next, we define the Koopman operator of this system.

Definition 1: Let $F : \mathcal{X} \to \mathcal{X}$ be a continuous map. The *Koopman operator* of F is the linear operator $\mathcal{K} : \mathcal{F} \to \mathcal{F}$ defined by

$$\mathcal{K}f = f \circ F, \quad \forall\, f \in \mathcal{F},$$

i.e., $(\mathcal{K}f)(x) = f(F(x))$ for all $x \in \mathcal{X}$ and $f \in \mathcal{F}$.

Eigenvalues and eigenfunctions of this operator are defined in the traditional way:

Definition 2: Let $\mathcal{K} : \mathcal{F} \to \mathcal{F}$ be a linear operator. A non-zero function $g \in \mathcal{F}$ is an *eigenfunction* of $\mathcal{K}$ if there exists $\lambda \in \mathbb{C}$, called an *eigenvalue*, such that $\mathcal{K}g = \lambda g$.

The connections between the eigenvalues and eigenfunctions of the Koopman operator and the properties of the dynamical system (such stability, chaos, and ergodicity) have been extensively studied [2], [3]. We mention below two results that we will use in the numerical experiments.

Proposition 1 ([2, §3.A]): Let $F : \mathcal{X} \to \mathcal{X}$ be a continuous map and $\mathcal{K}$ its Koopman operator. Let $x_* \in \mathcal{X}$ be an asymptotically stable equilibrium point of F. Let $g \in \mathcal{F}$ be an eigenfunction of $\mathcal{K}$ with eigenvalue 1. Assume that g is continuous at x_*. Then, g is constant on the basin of attraction of x_*.

Proposition 2 ([2, §3.A]): Let $F : \mathcal{X} \to \mathcal{X}$ be a continuous map and $\mathcal{K}$ its Koopman operator. Let $x_* \in \mathcal{X}$ be a globally asymptotically stable equilibrium point of F.[3] Define the set $\mathcal{F}_0 = \{f \in \mathcal{F} : f(x_*) = 0\}$. It holds that:

i) $\mathcal{F}_0$ is invariant for $\mathcal{K}$, i.e., $\mathcal{K}\mathcal{F}_0 \subseteq \mathcal{F}_0$.

Let $g \in \mathcal{F}_0$ be an eigenfunction of $\mathcal{K}$ with eigenvalue $\lambda \in \mathbb{C}$. Assume that g is continuous at x_0. It holds that:

ii) $|\lambda| < 1$.

iii) $x \mapsto |g(x)|$ provides a *semi-Lyapunov function* ("semi" in the sense that it may be 0 at $x \neq x_*$) for F.

Projected Koopman operator

As mentioned earlier, working in an infinite-dimensional space of observables is typically not tractable computationally. Therefore, we usually restrict our attention to a finite-dimensional subspace of observables, denoted $\mathcal{D} \subseteq \mathcal{F}$. Since it is not guaranteed that $\mathcal{D}$ is invariant for $\mathcal{K}$, we need to project back on $\mathcal{D}$ after application of $\mathcal{K}$. Given a closed linear subspace $\mathcal{D} \subseteq \mathcal{F}$, the *projection* on $\mathcal{D}$ is the linear operator $\Pi_{\mathcal{D}} : \mathcal{F} \to \mathcal{F}$ defined by

$$\Pi_{\mathcal{D}} f = \operatorname*{arg\,min}_{g \in \mathcal{D}} \|f - g\|. \tag{2}$$

The *projected Koopman operator* on $\mathcal{D}$ is then defined as

$$K_{\mathcal{D}} : \mathcal{D} \to \mathcal{D}, \quad K_{\mathcal{D}} = \Pi_{\mathcal{D}} \circ \mathcal{K}.$$

In the following, we assume that $\mathcal{D}$ is a finite-dimensional linear subspace of the Hilbert space $\mathcal{F}$, and we denote its dimension by D (typically, $D \gg 1$). Every finite-dimensional linear subspace of a normed space is closed; hence $\mathcal{D}$ is closed and the orthogonal projection $\Pi_{\mathcal{D}}$ in (2) is well defined and linear. We also assume that $\mathcal{K}\mathcal{D} \subseteq \mathcal{F}$.[4] With these assumptions, $K_{\mathcal{D}}$ is well defined and is a finite-dimensional linear operator. We denote its eigenvalues (counted with algebraic multiplicity) by $\lambda_1, \dots, \lambda_D \in \mathbb{C}$, ordered as

$$|\lambda_1| \geq |\lambda_2| \geq \cdots \geq |\lambda_D|. \tag{3}$$

The goal of this paper is to approximate the m dominant eigenvalues and associated invariant subspace of the *projected* operator $K_{\mathcal{D}}$, for some $1 \leq m \ll D$, without computing $K_{\mathcal{D}}$ explicitly. Thus, the primary object targeted by our power iteration is $K_{\mathcal{D}} = \Pi_{\mathcal{D}} \circ \mathcal{K}$, rather than the infinite-dimensional Koopman operator $\mathcal{K}$ itself (the

[2] That is, $\mathcal{F}$ consists of the Σ-measurable functions $f : \mathcal{X} \to \mathbb{C}$ such that $\|f\|^2 \triangleq \mathbb{E}_\mu[|f|^2] < \infty$.

[3] The assumption of *global* attractivity can be made without loss of generality since it suffices to restrict $\mathcal{X}$ to the basin of attraction of x_*.

[4] This is the case for instance if $\mathcal{D}$ contains only piecewise continuous functions and F is a continuous map.

relation between eigenfunctions of $K_{\mathcal{D}}$ and those of $\mathcal{K}$ is a separate approximation issue and can be affected by spectral pollution, as recalled in Remark 1 below). We introduce in the next section a power-iteration-based algorithm that does not require an explicit matrix representation of $K_{\mathcal{D}}$ (nor an explicit basis of $\mathcal{D}$, e.g., when the projection is implemented through neural regression). Furthermore, this approach can be implemented in a fully data-driven way.

Remark 1: As mentioned in the introduction, the projection of the Koopman operator may induce *spectral pollution*, i.e., eigenfunctions of $K_{\mathcal{D}}$ may not be good approximations of eigenfunctions of $\mathcal{K}$ [11]. We stress that this is an issue common to all Koopman methods based on projection on a finite-dimensional subspace [10], [12]–[14], [16]–[19]. Post-processing techniques can be used to filter out spurious eigenfunctions [11]; we will use these techniques in our numerical experiments (Section IV) to compare the accuracy of different methods.

III. Power-iteration-based algorithm

We present and analyze our power-iteration-based algorithm to compute the dominant eigenvalues of $K_{\mathcal{D}}$, which constitutes the main theoretical contribution of this work. We first present its theoretical *idealized* version (which is rather straightforward), then a practical neural data-driven version. We discuss the soundness of the latter, based on the neural tangent kernel theorem.

A. Idealized algorithm

This idealized version is not applicable in most practical situations, but nevertheless conveys the main theoretical ideas behind the algorithm. The base idea is to simply apply the power iteration algorithm on the operator $K_{\mathcal{D}}$. The power iteration algorithm consists in choosing an initial set of linearly independent functions $\{f_1^{(0)}, \dots, f_m^{(0)}\} \subseteq \mathcal{D}$, and applying recursively $K_{\mathcal{D}}$ to each of these functions. Possibly, a normalization step can be applied regularly to avoid ill-conditioning. A pseudo-code is presented in Algorithm 1.

Algorithm 1: Idealized Koopman power iteration

Data: Koopman operator $\mathcal{K}$, subspace $\mathcal{D} \subseteq \mathcal{F}$ and projection $\Pi_{\mathcal{D}}$, # modes $m \in \mathbb{N}_{>0}$

1 $\{f_j^{(0)}\}_{j=1}^m \leftarrow$ select m lin. ind. functions from $\mathcal{D}$

2 **for** $k = 0, 1, 2, \dots$ **do**

3 $\quad \{h_j^{(k+1)}\}_{j=1}^m \leftarrow \{\mathcal{K} f_j^{(k)}\}_{j=1}^m$

4 $\quad \{g_j^{(k+1)}\}_{j=1}^m \leftarrow \{\Pi_{\mathcal{D}} h_j^{(k+1)}\}_{j=1}^m$

5 $\quad \{f_j^{(k+1)}\}_{j=1}^m \leftarrow \mathrm{OrthoBasis}(\{g_j^{(k+1)}\}_{j=1}^m)$

`// Extract orthonormal basis if wanted`

Classical results from numerical linear algebra guarantees convergence of the power iteration algorithm toward the dominant modes under assumption of separation of the modulus of the mth and $(m+1)$st eigenvalues.

Proposition 3 ([20, Th. 7.3.1]): Consider a Koopman operator $\mathcal{K}$ and a finite-dimensional subspace $\mathcal{D} \subseteq \mathcal{F}$. Let the eigenvalues of $K_{\mathcal{D}}$ be ordered as in (3). Assume that $|\lambda_m| > |\lambda_{m+1}|$. Let $\mathcal{V}_m \subseteq \mathcal{D}$ be the invariant subspace of $K_{\mathcal{D}}$ associated to the first m eigenvalues. Consider an execution of Algorithm 1. Then, for almost all[5] choices of $\{f_1^{(0)}, \dots, f_m^{(0)}\} \subseteq \mathcal{D}$ in line 1, $\mathrm{span}\{f_1^{(k)}, \dots, f_m^{(k)}\}$ converges towards $\mathcal{V}_m$ when $k \to \infty$.

B. Neural data-driven algorithm

The purpose of the neural data-driven version is to relax the computation of $\mathcal{K} f_j$ and $\Pi_{\mathcal{D}} h_j$ in Algorithm 1, which are both intractable for most practical problems.[6]

Therefore, the projection $\Pi_{\mathcal{D}}$ is first replaced by a data-driven approximation. Given N samples $\{x_i\}_{i=1}^N \subseteq \mathcal{X}$ drawn i.i.d. from μ, we approximate $\langle \cdot, \cdot \rangle$ and $\|\cdot\|$ by Monte–Carlo approximations $\langle \cdot, \cdot \rangle'_N$ and $\|\cdot\|'_N$:

$$\langle f, g \rangle'_N \triangleq \frac{1}{N} \sum_{i=1}^N f(x_i) \bar{g}(x_i), \quad \|f\|'^2_N \triangleq \langle f, f \rangle'_N. \tag{4}$$

This allows to approximate $\Pi_{\mathcal{D}}$ by replacing "$\|\cdot\|$" by "$\|\cdot\|'_N$" in (2). The resulting operator is denoted by $\Pi'_{N,\mathcal{D}}$:[7]

$$\Pi'_{N,\mathcal{D}} f = \operatorname*{arg\,min}_{g \in \mathcal{D}} \|f - g\|'_N. \tag{5}$$

When the elements of $\mathcal{D}$ are represented by a neural network ϕ^θ, the argmin in (5) reduces to classical function regression using ℓ^2 loss:

$$\Pi'_{N,\Phi} f = \operatorname*{arg\,min}_{\theta \in \mathbb{R}^{n_\theta}} \frac{1}{N} \sum_{i=1}^N \left| \phi^\theta(x_i) - f(x_i) \right|^2 \tag{6}$$

(we argue in Section III-C that this yields a linear operator under appropriate assumptions). (6) can be solved by using gradient descent, Adam, AdamW, etc. [28].

Following Algorithm 1, $\Pi'_{N,\Phi}$ must be applied to $h_j^{(k+1)} = \mathcal{K} f_j^{(k)}$. Hence, $f(x_i)$ in (6) must be replaced by $h_j^{(k+1)}(x_i)$. Since $\mathcal{K} f_j(x_i) = f_j(F(x_i))$, it suffices to compute

$$x_i^+ \triangleq F(x_i) \quad i = 1, \dots, N, \tag{7}$$

and let $h_j^{(k+1)}(x_i) = f_j^{(k)}(x_i^+)$. In particular, if the current iterate is $f_j^{(k)} = \phi^{\theta_j^{(k)}}$, the next pre-orthogonalization iterate is given by $g_j^{(k+1)} = \phi^{\zeta_j^{(k+1)}}$ where

$$\zeta_j^{(k+1)} \in \operatorname*{arg\,min}_{\theta \in \mathbb{R}^{n_\theta}} \frac{1}{N} \sum_{i=1}^N \left| \phi^\theta(x_i) - \phi^{\theta_j^{(k)}}(x_i^+) \right|^2. \tag{8}$$

(8) makes explicit that no representation of $\mathcal{K}$ is constructed: at each power iteration, a neural network is simply regressed on the values of the previous iterate evaluated at the successor samples. Hence, the computations are fully data-driven, based only on i.i.d. samples $\{x_i\}_{i=1}^N \subseteq \mathcal{X}$ and

[5] With respect to the natural Lebesgue measure on $\mathcal{D}^m$, which means the Lebesgue measure than can be derived from any orthonormal basis of $\mathcal{D}$.

[6] $\mathcal{K} f_j$ requires knowing f_j at every x. $\Pi_{\mathcal{D}} h_j$ requires either an orthonormal basis + numerical integration or a basis + numerical integration + the inversion of a $D \times D$ Gram matrix [13].

[7] We assume without loss of generality (possibly at the cost of taking more samples) that $\|\cdot\|'_N$ is a norm on $\mathcal{D}$, so that the argmin in (5) is unique.

their successors $\{x_i^+\}_{i=1}^N$ by the system as in (7). A pseudo-code of the neural data-driven algorithm is presented in Algorithm 2.

Algorithm 2: Neural data-driven Koopman power iteration

Data: Samples $\{x_i\}_{i=1}^N \subseteq \mathcal{X}$ and successors $\{x_i^+\}_{i=1}^N$, neural network ϕ^θ, # modes $m \in \mathbb{N}_{>0}$

1 $\{\theta_j^{(0)}\}_{j=1}^m \leftarrow$ initialize m param. vec. from $\mathbb{R}^{n_\theta}$
2 **for** $k = 0, 1, 2, \ldots$ **do**
3 **for** $j = 1, \ldots, m$ **do**
4 $\zeta_j^{(k+1)} \leftarrow$ solve regression (8)
5 $g_j^{(k+1)} \leftarrow \phi^{\zeta_j^{(k+1)}}$
6 $\{\theta_j^{(k+1)}\}_{j=1}^m \leftarrow \mathrm{OrthoBasis}(\{g_j^{(k+1)}\}_{j=1}^m)$
  `// cf. Remark 2`

Remark 2: The orthonormalization of the basis of neural networks (line 6 in Algorithm 2) can be done easily from the samples as follows:

1) At a given iteration k, compute the vectors $v_1, \ldots, v_m \in \mathbb{C}^N$ defined by $v_j = [g_j^{(k+1)}(x_i)]_{i=1}^N$.
2) Compute an orthonormal basis $\{q_1, \ldots, q_m\}$ of $\{v_1, \ldots, v_m\}$ (e.g., by computing the QR decomposition of the matrix $[v_1, \ldots, v_m] \in \mathbb{C}^{N \times m}$ [20]).
3) For all $1 \leq j \leq m$, find

$$\theta_j^{(k+1)} \in \operatorname*{arg\,min}_{\theta \in \mathbb{R}^{n_\theta}} \frac{1}{N} \sum_{i=1}^N \left| \phi^\theta(x_i) - q_j[i] \right|^2 .$$

If the fitting error in 3) is less than or equal to ϵ, it follows that for all $1 \leq j_1, j_2 \leq m$, $|\langle \phi^{\theta_{j_1}}, \phi^{\theta_{j_2}} \rangle'_N - \delta_{j_1,j_2}| \leq 2\epsilon + O(\epsilon^2)$. In other words, $\{\phi^{\theta_1}, \ldots, \phi^{\theta_m}\}$ is ϵ-orthonormal with respect to $\langle \cdot, \cdot \rangle'_N$.[8] Furthermore, under the assumption that the set of functions parametrized by ϕ^θ forms a linear subspace (see Section III-C), the global minimum of fitting error in 3) achieves $\epsilon = 0$. ◁

C. Analysis of neural data-driven v.s. idealized

We analyze the neural data-driven algorithm, in particular its soundness to approximate the idealized algorithm.

First, we start with the Monte–Carlo approximation of the norm and inner product. By the strong law of large numbers, we have that both approximations are sound as $N \to \infty$:

Theorem 1 ([29, § 8]): Let $\{x_i\}_{i=1}^\infty \subseteq \mathcal{X}$ be a sequence of i.i.d. samples from μ. Let $f, g \in \mathcal{F}$. With probability 1 on the sampling of $\{x_i\}_{i=1}^\infty$, it holds that

$$\langle f, g \rangle'_N \to \langle f, g \rangle \quad \text{and} \quad \|f\|'_N \to \|f\|, \quad \text{as} \quad N \to \infty.$$

As a corollary, we obtain the main result of this section:

Corollary 1 (Main result): Consider a Koopman operator $\mathcal{K}$ and a neural network ϕ^θ. Assume that $\Pi'_{N,\Phi}$ in (6) is a linear operator with image set denoted by $\mathcal{D}$ (which is a linear subspace). Assume that $\mathcal{D} \subseteq \mathcal{F}$ and $\mathcal{K}\mathcal{D} \subseteq \mathcal{F}$. Let $\{x_i\}_{i=1}^\infty \subseteq \mathcal{X}$ be a sequence of i.i.d. samples from μ.

i) For any $f \in \mathcal{F}$, it holds that $\Pi'_{N,\Phi} \mathcal{K} f \to \Pi_{\mathcal{D}} \mathcal{K} f$ with probability one as $N \to \infty$.
ii) Consequently, for any fixed $k \in \mathbb{N}_{>0}$, the iterates $\{\phi^{\theta_1^{(k)}}, \ldots, \phi^{\theta_m^{(k)}}\}$ of Algorithm 2 converge with probability one to the iterates $\{f_1^{(k)}, \ldots, f_m^{(k)}\}$ of Algorithm 1 (with the same $\mathcal{D}$), as $N \to \infty$.

Remark 3 (Convergence and approximation limits): It is useful to distinguish the different limiting arguments underlying Algorithm 2. (i) At the population level, Algorithm 1 is a classical subspace iteration for the projected Koopman operator $K_{\mathcal{D}}$. Under the spectral-gap assumption $|\lambda_m| > |\lambda_{m+1}|$, its iterates satisfy

$$\operatorname{span}\{f_1^{(k)}, \ldots, f_m^{(k)}\} \longrightarrow \mathcal{V}_m \qquad \text{as } k \to \infty,$$

for almost every initialization. (ii) Algorithm 2 replaces each exact projection appearing in this ideal iteration by an empirical projection computed from N samples. Corollary 1 shows that, for any fixed number of iterations k, this approximation becomes exact as $N \to \infty$: the data-driven iterates converge almost surely to the corresponding ideal iterates. (iii) Finally, the neural implementation introduces an additional approximation, since the empirical projection is obtained by training a neural network. The NTK argument below shows that, in the infinite-width regime, this training procedure can be interpreted as a linear empirical projection, thereby connecting the neural implementation to the setting of Corollary 1.

Finally, we discuss the assumption that $\Pi'_{N,\Phi}$ is a linear operator, made in Corollary 1 (and Remark 2). Note that it is in general not a linear operator since $\{\phi^\theta : \theta \in \mathbb{R}^{n_\theta}\}$ is not a linear subspace.[9] Nevertheless, the *neural tangent kernel* (NTK) theorem [21], [30] allows us to assume that when the width of ϕ^θ is large, $\Pi'_{N,\Phi}$ yields a linear operator.

Theorem 2 ([21, Th. 2]): Consider a neural network ϕ^θ with random Gaussian parameter $\theta \in \mathbb{R}^{n_\theta}$. In the infinite-width limit, i.e., when the hidden widths $c_1, \ldots, c_L \to \infty$, the neural tangent kernel

$$\Theta_\theta(x, y) \triangleq \nabla_\theta \phi^\theta(x)^\top \nabla_\theta \phi^\theta(y), \quad x, y \in \mathcal{X},$$

converges in probability (with respect to θ) to a *deterministic*[10] kernel Θ_*. Moreover, when trained by gradient flow on the ℓ^2 loss (and with parameter θ initialized randomly as above), the network output evolves according to

$$\frac{\mathrm{d}}{\mathrm{d}t} \phi^{\theta_t}(x) \propto -\sum_{i=1}^N \Theta_*(x, x_i) \{\phi^{\theta_t}(x_i) - f(x_i)\}.$$

Hence, in the infinite-width regime, the training dynamics corresponds to *kernel regression* in the reproducing kernel Hilbert space associated with Θ_*.

[8] Note that exact orthonormality is not needed since our aim is merely to avoid ill-conditioning.

[9] For instance, it is not guaranteed in general that for all $\theta_1, \theta_2 \in \mathbb{R}^{n_\theta}$, there is $\theta \in \mathbb{R}^{n_\theta}$ such that $\phi^\theta = \phi^{\theta_1} + \phi^{\theta_2}$.

[10] Θ_* depends only on the depth L, the activation function and the variance of the parameter distribution.

The NTK theorem implies that, in the infinite-width regime and under gradient-flow training from the stated random initialization, the network output evolves in the linear span of the kernel sections $\Theta_*(\cdot, x_i)$ evaluated at the training inputs. Consequently, least-squares training acts as kernel regression in the finite-dimensional data-dependent space

$$\mathcal{D}_N = \operatorname{span}\{\Theta_*(\cdot, x_i)\}_{i=1}^N.$$

In this asymptotic regime, the fitted-value map $f \mapsto \Pi'_{N,\Phi} f$ can therefore be identified with the empirical linear projection $\Pi'_{N,\mathcal{D}_N}$ (up to the usual qualifications associated with initialization and optimization in the NTK limit). This is the sense in which the nonlinear neural parametrization induces the linear projection assumed in Corollary 1; it is not an assumption that finite-width neural networks themselves form a linear function class. Thus, Corollary 1 provides an asymptotic justification for the neural implementation.[11]

IV. Numerical experiments

See extended version [31].

V. Scope and limitations

See extended version [31].

VI. Conclusions

See extended version [31].

References

[1] B. O. Koopman, "Hamiltonian systems and transformation in Hilbert space," *Proceedings of the National Academy of Sciences*, vol. 17, no. 5, pp. 315–318, 1931.

[2] A. Mauroy and I. Mezić, "Global stability analysis using the eigenfunctions of the Koopman operator," *IEEE Transactions on Automatic Control*, vol. 61, no. 11, pp. 3356–3369, 2016.

[3] I. Mezić, "Spectrum of the Koopman operator, spectral expansions in functional spaces, and state-space geometry," *Journal of Nonlinear Science*, vol. 30, no. 5, pp. 2091–2145, 2020.

[4] ——, "Spectral properties of dynamical systems, model reduction and decompositions," *Nonlinear Dynamics*, vol. 45, no. 1, pp. 309–325, 2005.

[5] E. Kaiser, J. N. Kutz, and S. L. Brunton, "Data-driven discovery of Koopman eigenfunctions for control," *Machine Learning: Science and Technology*, vol. 2, no. 35023, pp. 1–32, 2021.

[6] M. Budišić, R. Mohr, and I. Mezić, "Applied Koopmanism," *Chaos: An Interdisciplinary Journal of Nonlinear Science*, vol. 22, no. 4, pp. 1–33, 2012.

[7] S. E. Otto and C. W. Rowley, "Koopman operators for estimation and control of dynamical systems," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, no. 1, pp. 59–87, 2021.

[8] S. L. Brunton, M. Budišić, E. Kaiser, and J. N. Kutz, "Modern Koopman theory for dynamical systems," *SIAM Review*, vol. 64, no. 2, pp. 229–340, 2022.

[9] M. J. Colbrook, Z. Drmač, and A. Horning, "An introductory guide to Koopman learning," 2025, arXiv preprint arXiv:2510.22002.

[10] M. J. Colbrook, "The multiverse of dynamic mode decomposition algorithms," in *Handbook of numerical analysis*, S. Mishra and A. Townsend, Eds. Elsevier, 2024, vol. 25, pp. 127–230.

[11] M. J. Colbrook and A. Townsend, "Rigorous data-driven computation of spectral properties of Koopman operators for dynamical systems," *Communications on Pure and Applied Mathematics*, vol. 77, pp. 221–283, 2024.

[12] P. J. Schmid, "Dynamic mode decomposition of numerical and experimental data," *Journal of Fluid Mechanics*, vol. 656, pp. 5–28, 2010.

[13] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, "A data–driven approximation of the Koopman operator: extending dynamic mode decomposition," *Journal of Nonlinear Science*, vol. 25, pp. 1307–1346, 2015.

[14] H. Arbabi and I. Mezic, "Ergodic theory, dynamic mode decomposition, and computation of spectral properties of the Koopman operator," *SIAM Journal on Applied Dynamical Systems*, vol. 16, no. 4, pp. 2096–2126, 2017.

[15] Q. Li, F. Dietrich, E. M. Bollt, and I. G. Kevrekidis, "Extended dynamic mode decomposition with dictionary learning: a data-driven adaptive spectral decomposition of the Koopman operator," *Chaos: An Interdisciplinary Journal of Nonlinear Science*, vol. 27, no. 10, pp. 1–10, 2017.

[16] N. Takeishi, Y. Kawahara, and T. Yairi, "Learning Koopman invariant subspaces for dynamic mode decomposition," in *NIPS'17: Proceedings of the 31st International Conference on Neural Information Processing System*. ACM, 2017, pp. 1130–1140.

[17] B. Lusch, J. N. Kutz, and S. L. Brunton, "Deep learning for universal linear embeddings of nonlinear dynamics," *Nature communications*, vol. 9, no. 4950, pp. 1–10, 2018.

[18] D. J. Alford-Lago, C. W. Curtis, A. T. Ihler, and O. Issan, "Deep learning enhanced dynamic mode decomposition," *Chaos: An Interdisciplinary Journal of Nonlinear Science*, vol. 32, no. 3, pp. 1–14, 2022.

[19] A. K. Mondal, S. S. Panigrahi, S. Rajeswar, K. Siddiqi, and S. Ravanbakhsh, "Efficient dynamics modeling in interactive environments with Koopman theory," 2023, arXiv preprint arXiv:2306.11941.

[20] G. H. Golub and C. F. Van Loan, *Matrix computations*, 4th ed. Baltimore, MD: The Johns Hopkins University Press, 2013.

[21] A. Jacot, F. Gabriel, and C. Hongler, "Neural tangent kernel: convergence and generalization in neural networks," in *NIPS'18: Proceedings of the 32nd International Conference on Neural Information Processing Systems*. ACM, 2018, pp. 8580–8589.

[22] P. Bevanda, S. Sosnowski, and S. Hirche, "Koopman operator dynamical models: learning, analysis and control," *Annual Reviews in Control*, vol. 52, pp. 197–212, 2021.

[23] D. Ha and J. Schmidhuber, "World models," 2018, arXiv preprint arXiv:1803.10122.

[24] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. IEEE, 2023, pp. 15 619–15 629.

[25] J. Ulmen, G. Sundaram, and D. Görges, "Learning state-space models of dynamic systems from arbitrary data using joint embedding predictive architectures," *IFAC-PapersOnLine*, vol. 59, no. 18, pp. 19–24, 2025.

[26] B. Terver, R. Balestriero, M. Dervishi, D. Fan, Q. Garrido, T. Nagarajan, K. Sinha, W. Zhang, M. Rabbat, Y. LeCun, and A. Bar, "A lightweight library for energy-based joint-embedding predictive architectures," 2026, arXiv preprint arXiv:2602.03604.

[27] A. Friedman, *Foundations of modern analysis*. New York, NY: Dover Publications, 1982.

[28] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, arXiv preprint arXiv:1711.05101.

[29] K. B. Athreya and S. N. Lahiri, *Measure theory and probability theory*. New York, NY: Springer, 2006.

[30] E. Golikov, E. Pokonechnyy, and V. Korviakov, "Neural tangent kernel: a survey," 2022, arXiv preprint arXiv:2208.13614.

[31] G. O. Berger and R. M. Jungers, "Data-driven Koopman mode approximation: A neural power iteration algorithm," 2026, arXiv preprint arXiv:2608.26943.

[11] We omit the verification that $\mathcal{D} \subseteq \mathcal{F}$ and $\mathcal{K}\mathcal{D} \subseteq \mathcal{F}$ because this is expected to hold generically since neural networks are piecewise smooth.